COS 350 Lecture 4

unix text file viewing:

  cat, head, tail, more, less

case study: cat

buffered files

file streams

case study: more

/dev/tty

Unix commands for viewing text files

`cat` (concatenate) output file(s) to terminal

`head` show first 10 lines of file

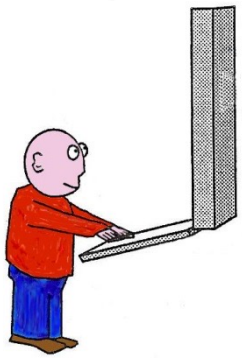
`tail` show last 10 lines of file

`more` pager shows 1 page at a time

commands

- **`space`** scroll 1 page
- **`Enter`** scroll 1 line
- **`b`** scroll Backwards 1 page
- /**`pattern`** search for pattern
- **`q`** quit
- **`h`** help

`less` pager like more but has more commands



demo commands

man more

<u>Understand Unix/Linux Programming</u>

Book is organized around case studies

Q1. **What does that do?**

look at real programs

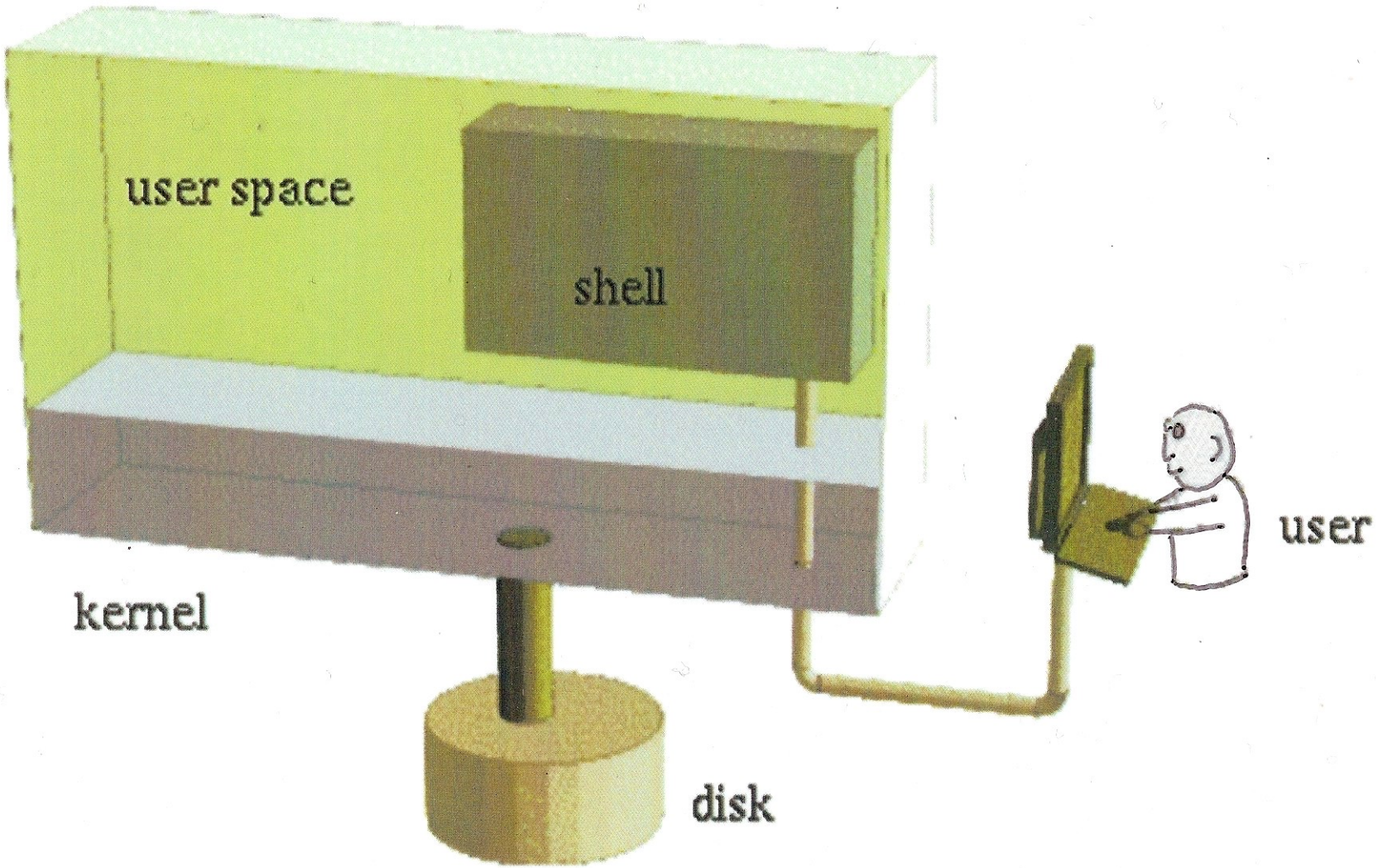
Q2. **How does that work?**

learn about C libraries and Unix system calls

Q3. **Can I try to do that?**

write our own version

login, run programs, logout



My case study: **cat**

Q1: What does that do?

cat file1	*read file and send to screen*
cat file1 file2 > files3	*concatenates files and writes to file3*
cat > file4	*send keyboard input to file4*
cat -n file3	*number lines on output*

Q2: How does that work? → learn about files

in <stdio.h> there is a type **FILE** *functions on FILE's are buffered*

FILE *fp; *declares a file pointer*

FILE ***fopen**(char *filename, char *mode);
returns NULL if error

"r" = reading existing file
"w" = writing (create or truncate)
"a" = append (create or append to)

int **fclose**(FILE *fp);
returns 0 if ok, EOF if error

int **fflush**(FILE *fp); *force out any buffered output*
returns 0 if ok, EOF if error

input:

```
int fgetc(FILE *stream);
int getc(FILE *stream);
int getchar(void);
int ungetc(int c, FILE *stream);

char *fgets(char *s, int n, FILE *stream);
```

~~char *gets(char *s);~~

output:

```
int fputc(int c, FILE *stream);
int putc(int c, FILE *stream);
int putchar(int c);

int puts(char *s);
int fputs(char *s, FILE *stream);

int fprintf(FILE *stream, char *format, /* args*/ ...);
```

print into string buffer:

```
int sprintf(char *s, char *format, /* args*/ ...);
```

Redirection and Pipes

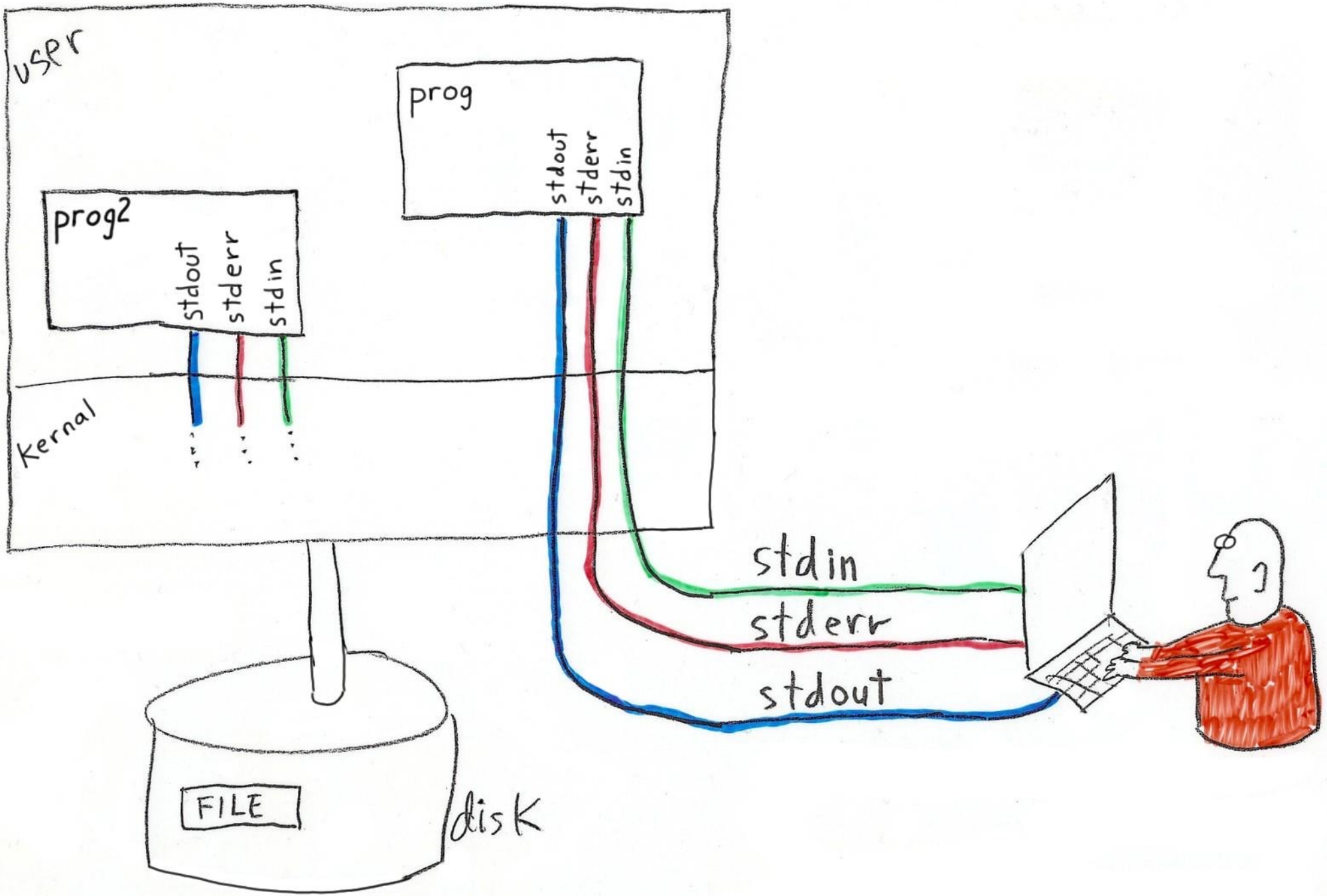
cat file1 file2 > file3

this is redirection that sends output of the cat program to file file3

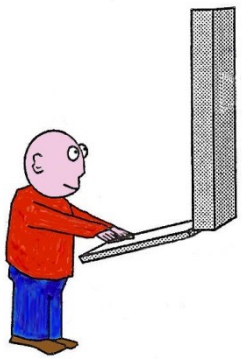
cat file1 file2 | wc

this pipes the output of the cat into the input the program wc

The shell handles all redirection and pipes!
The cat program does not even know that it is occurring!


usr
prog
stdout
stderr
stdin
prog2
stdout
stderr
stdin
Kernal
stdin
stderr
stdout
FILE
disK

Q3: Can I do that?



mycat.c
demo
? cat binary file
? buffer overflow

Book's case study: **more**

Q1: What does that do?

more mlk_speech	*1. show file 1 page at a time*
	2. print prompt in reverse video
	3. keyboard commands
cat –n mlk_speech | more	*4. input piped from another program*

Q2: How does that work?

1. *show file 1 page at a time*
   loop 23 times fgets() & fputs()

2. *print prompt in reverse video*
   magic terminal control characters

3. *keyboard commands*
   c = getchar() read keyboard commands from stdin

4. *input piped from another program*
   read file from stdin
   read keyboard commands from /dev/tty

1. show file 1 page at a time

```
#define  PAGELEN  23    /* leave room for prompt */
#define  LINELEN  512

/* read PAGELEN lines,
 * then call see_more() for further instructions
 */
void do_more( FILE *fp ) {
   char line[LINELEN];
   int num_of_lines = 0;
   int reply;

   while ( fgets( line, LINELEN, fp ) ){  /* more input */
      if ( num_of_lines == PAGELEN ) {    /* full screen? */
         reply = see_more();              /* y: ask user  */
         if ( reply == 0 )                /* n: done   */
            break;
         num_of_lines -= reply;           /* reset count */
      }
      if ( fputs( line, stdout ) == EOF ) /* show line */
         exit(1);                         /* or die   */
      num_of_lines++;                     /* count it */
   }
}
```

2. print prompt in reverse video

```
/* print message, wait for response,
 * return # of lines to advance
 * q means no, space means yes, CR means one line
 */
int see_more() {
   int c;

   printf("\033[7m more? \033[m");  // reverse on a vt100
   while( (c=getchar()) != EOF ) {  // get response from stdin
     if ( c == 'q' )                // q -> N
       return 0;                    // 0 will cause quit
     if ( c == ' ' )                // ' ' => next page
       return PAGELEN;              // how many to show
     if ( c == '\n' )               // Enter key => 1 line
       return 1;
   }
   return 0;
}
```

3. keyboard commands

```
/* print message, wait for response,
 * return # of lines to advance
 * q means no, space means yes, CR means one line
 */
int see_more() {
   int c;

   printf("\033[7m more? \033[m");  // reverse on a vt100
   while( (c=getchar()) != EOF ) {  // get response from stdin
     if ( c == 'q' )                // q -> N
       return 0;                    // 0 will cause quit
     if ( c == ' ' )                // ' ' => next page
       return PAGELEN;              // how many to show
     if ( c == '\n' )               // Enter key => 1 line
       return 1;
   }
   return 0;
}
```

4. input piped from another program

```
void do_more( FILE *fp ) {
  char line[LINELEN];
  int  num_of_lines = 0, reply;
  FILE *fp_tty;

  fp_tty = fopen("/dev/tty", "r" );   // cmd stream
  if ( fp_tty == NULL )               // if open fails
    exit(1);                          // no use in running

  while ( fgets( line, LINELEN, fp ) ){     // more input
    if ( num_of_lines == PAGELEN ) {  // full screen?
      reply = see_more(fp_tty);       // pass FILE cmd stream
      if ( reply == 0 )               //    n: done
```

```
int see_more(FILE *cmdStream) {
  int c;
  printf("\033[7m more? \033[m");   // reverse on a vt100
  while((c=getc(cmdStream))!=EOF) { // reads from tty
    if ( c == 'q' )                 // q -> N
      return 0;
```